

Parallel Scaling

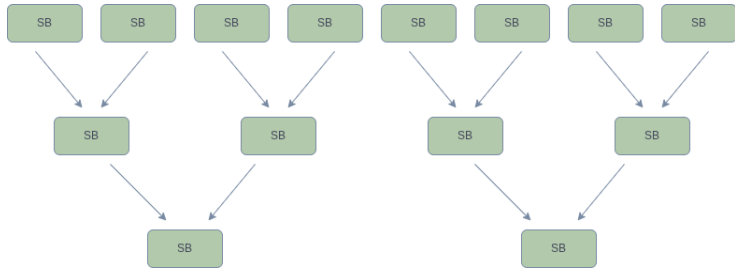
Ana Costa Pereira, Joshua Davies

26/06/2026



- ① Parallel scalings
- ② SortBotMerge
- ③ Addition of new SortBot

- Parallel scalings: scaling of the tests before the changes at different thread count, screenshots of the distribution of work between main thread and working threads from profiler, comment on performance of different tests



- Running in parallel: Sortbots send terms between each other and to the master

Working of the code:

- Terms are allowed to overlap between blocks;
- Reading thread always locks pair of locks - the current thread and the previous thread;
- 0th block - the overlap in the end of term is copied to 0th so that the terms are contiguous - this is no longer needed when we impose that only full terms can fit into the blocks

Before changes

- Default buffer sizes were increased over time
- Size of sortbots increased;
- The output from a worker fits in a single block → no advantage on running in parallel;
- Each stage of sortbot merge tree runs one after the other
- Terms can be partially shared between blocks: when a term is being written in a block and the block finishes, the term keeps being written into the next block

After changes

- Blocks are made smaller
- Whole terms can only be written in one block: if it does not fit, it goes into the next block
- Workers that fill the blocks try to detect if there is a reader waiting for them
- Obtain a lock of the block before filling the block; if not, it means the reading thread already has the lock which means that it is processing the block which means it wants small terms from the current filling block, so we move on to the next block in writing thread
- There is always a minimum number of terms in the block - MINIMUMNUMBEROFTERMS

Some results after the changes

- There is no decrease in performance for any tests
- There is a performance improvement for benchmarks where term merging is expensive (PolyRatFun, PolyRatFunExpand)
- Forcer, mbox1l

- SortBotMerge: Explain the changes introduced -
NUMBEROFBLOCKSINSERT,
MINMUMNUMBEROSTERMS and final choice of parameters
- How did we change the code? Explin logic of the main changes
- Show results for different tests - improvements in scaling -
plots and also profiler screenshots for the tests that are most
affected

- Explain how the sortbots and main thread split the work as they work in parallel - draw tree diagram with sortbots as example
- Explain how we introduce a new sortbot and how that changes the splitting of the work, especially for the master, which now will be only writing
- Explain logic of the changes made in the code
- Explain results for the current benchmarks - not a significant difference because wither most time is spent in sorting or in writing, so the changes do not impact a lot
- Introduce new test where the splitting between sorting and writing is more equal, and show screenshot fro profiler
- add some results from running this in higher number of threads